Name: Gunjan Chandnani **Roll No: 13** **Div: D20A** **Lab: Blockchain**

Experiment 3

AIM: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

1. Blockchain Overview

A **blockchain** is a distributed, decentralized, and cryptographically secured digital ledger used to record transactions in a chronological and immutable manner. Unlike traditional centralized databases that depend on a trusted third party, blockchain operates on a **peer-to-peer network** where trust is achieved through cryptographic techniques and consensus protocols.

Each block in a blockchain contains:

- A set of verified transactions
- A timestamp indicating when the block was created
- The hash of the previous block, which links blocks together
- A nonce used during the mining process
- The block's own cryptographic hash

Blocks are connected using cryptographic hashes, forming a chain-like structure. Any attempt to modify data in a block changes its hash, which invalidates all subsequent blocks. This property ensures **immutability, data integrity, and tamper resistance**. Since copies of the blockchain are stored across multiple nodes, the system also provides **transparency, fault tolerance, and decentralization**.

---

2. Mining

Mining is the process by which new blocks are created, validated, and added to the blockchain. It plays a vital role in maintaining the **security, reliability, and decentralization** of the blockchain network.

Mining performs two main functions:

1. Verification and validation of transactions
2. Protection of the blockchain from malicious attacks

Mining is based on the **Proof-of-Work (PoW)** algorithm. In PoW, miners compete to solve a computationally complex mathematical puzzle. This involves repeatedly modifying a value

known as a **nonce** and recalculating the block's hash until it satisfies the network's difficulty condition, such as producing a hash with a specific number of leading zeros.

The difficulty level adjusts dynamically to control how frequently blocks are mined. Once a miner finds a valid hash, the block is broadcast to the network for verification. Due to the high computational cost involved, Proof-of-Work makes attacks like double spending and data manipulation extremely difficult.

---

3. Multi-Node Blockchain Network

A **multi-node blockchain network** consists of multiple independent nodes that participate in transaction validation and block propagation. In this system, three nodes are deployed on different ports (5001, 5002, and 5003) to simulate a real-world decentralized environment.

Each node in the network:

- Operates independently
- Maintains a complete copy of the blockchain
- Communicates with peer nodes using a peer-to-peer protocol

This architecture eliminates the need for a central server and ensures **decentralization, data redundancy, and high availability**. Even if one node fails or behaves maliciously, the blockchain network continues to operate correctly. This setup clearly demonstrates how blockchain systems maintain consistency and reliability in distributed environments.

---

4. Consensus Mechanism

A **consensus mechanism** is a protocol that allows all nodes in a blockchain network to agree on a single, consistent version of the ledger. Since nodes operate independently and blocks may be mined simultaneously, temporary inconsistencies or forks can occur.

In this system, the **Longest Chain Rule** is used as the consensus mechanism. According to this rule:

- The blockchain with the greatest number of blocks is considered valid
- The longest chain represents the highest cumulative computational effort
- Nodes discard shorter chains and adopt the longest valid chain

This mechanism ensures **eventual consistency**, prevents permanent forks, and protects the blockchain from double-spending attacks. Consensus mechanisms are essential for maintaining trust and integrity in decentralized systems.

---

5. Transactions and Mining Reward

A **transaction** is a digitally signed record that represents the transfer of value between participants in a blockchain network. Each transaction contains:

- Sender's address
- Receiver's address
- Transaction amount

Transactions are verified using cryptographic methods and temporarily stored in a transaction pool until they are included in a block.

To motivate miners, the blockchain uses a **mining reward mechanism**. When a block is successfully mined, a special transaction called a **coinbase transaction** is created. This transaction generates new cryptocurrency and rewards it to the miner. The reward system encourages honest participation and helps maintain network security.

---

6. Chain Replacement

Chain replacement is a synchronization mechanism that ensures all nodes in the blockchain network maintain a consistent and updated ledger. Due to network delays or simultaneous block creation, nodes may temporarily have different versions of the blockchain.

When the `/replace_chain` endpoint is invoked:

1. A node requests blockchain data from all peer nodes
2. The received chains are validated for correctness and integrity
3. The node replaces its own chain if a longer and valid chain is found

This mechanism resolves conflicts, removes forks, and ensures that all nodes converge to a single authoritative blockchain. Chain replacement is essential for maintaining decentralization without the need for central coordination.

Code:

node.py:

```
import datetime
import hashlib
import json
from flask import Flask, jsonify, request
import requests
from uuid import uuid4
from urllib.parse import urlparse

# ---------------- Blockchain Class ----------------
class Blockchain:
    def __init__(self):
        self.chain = []
        self.transactions = []
        self.nodes = set()
        self.create_block(proof=1, previous_hash='0')  # Genesis block

    def create_block(self, proof, previous_hash):
        block = {
            'index': len(self.chain) + 1,
            'timestamp': str(datetime.datetime.now()),
            'proof': proof,
            'previous_hash': previous_hash,
            'transactions': self.transactions
        }
        self.transactions = []   # clear pending transactions
        self.chain.append(block)
        return block

    def get_previous_block(self):
        return self.chain[-1]

    def proof_of_work(self, previous_proof):
        new_proof = 1
        while True:
            hash_operation = hashlib.sha256(
                str(new_proof**2 - previous_proof**2).encode()
            ).hexdigest()
            if hash_operation[:4] == '0000':
```

```
                return new_proof
            new_proof += 1

    def hash(self, block):
        encoded_block = json.dumps(block, sort_keys=True).encode()
        return hashlib.sha256(encoded_block).hexdigest()

    def is_chain_valid(self, chain):
        previous_block = chain[0]
        index = 1

        while index < len(chain):
            block = chain[index]

            if block['previous_hash'] != self.hash(previous_block):
                return False

            proof = block['proof']
            previous_proof = previous_block['proof']
            hash_operation = hashlib.sha256(
                str(proof**2 - previous_proof**2).encode()
            ).hexdigest()

            if hash_operation[:4] != '0000':
                return False

            previous_block = block
            index += 1

        return True

    def add_transaction(self, sender, receiver, amount):
        self.transactions.append({
            'sender': sender,
            'receiver': receiver,
            'amount': amount
        })
        return self.get_previous_block()['index'] + 1

    def add_node(self, address):
```

```python
        parsed_url = urlparse(address)
        self.nodes.add(parsed_url.netloc)

    def replace_chain(self):
        network = self.nodes
        longest_chain = None
        max_length = len(self.chain)

        for node in network:
            response = requests.get(f'http://{node}/get_chain')
            if response.status_code == 200:
                length = response.json()['length']
                chain = response.json()['chain']

                if length > max_length and self.is_chain_valid(chain):
                    max_length = length
                    longest_chain = chain

        if longest_chain:
            self.chain = longest_chain
            return True

        return False


# ---------------- Flask App ----------------
app = Flask(__name__)
node_address = str(uuid4()).replace('-', '')
blockchain = Blockchain()


@app.route('/mine_block', methods=['GET'])
def mine_block():
    previous_block = blockchain.get_previous_block()
    proof = blockchain.proof_of_work(previous_block['proof'])
    previous_hash = blockchain.hash(previous_block)

    # mining reward
    blockchain.add_transaction(node_address, 'Richard', 1)
```

```
    block = blockchain.create_block(proof, previous_hash)
    return jsonify(block), 200


@app.route('/get_chain', methods=['GET'])
def get_chain():
    return jsonify({
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }), 200


@app.route('/is_valid', methods=['GET'])
def is_valid():
    return jsonify({
        'valid': blockchain.is_chain_valid(blockchain.chain)
    }), 200


@app.route('/add_transaction', methods=['POST'])
def add_transaction():
    data = request.get_json()
    required = ['sender', 'receiver', 'amount']

    if not data or not all(k in data for k in required):
        return "Missing fields", 400

    index = blockchain.add_transaction(
        data['sender'],
        data['receiver'],
        data['amount']
    )

    return jsonify({
        'message': f'Transaction added to block {index}'
    }), 201


@app.route('/connect_node', methods=['POST'])
def connect_node():
```

```
    data = request.get_json()
    nodes = data.get('nodes')

    if nodes is None:
        return "No nodes", 400

    for node in nodes:
        blockchain.add_node(node)

    return jsonify({
        'nodes': list(blockchain.nodes)
    }), 201


@app.route('/replace_chain', methods=['GET'])
def replace_chain():
    replaced = blockchain.replace_chain()
    return jsonify({
        'replaced': replaced,
        'chain': blockchain.chain
    }), 200


# ---------------- Run Node ----------------
app.run(host='0.0.0.0', port=5001)
```

node2.py:
Copy `node.py`
Rename to `node2.py`
Change last line:
app.run(host='0.0.0.0', port=5002)

Output:

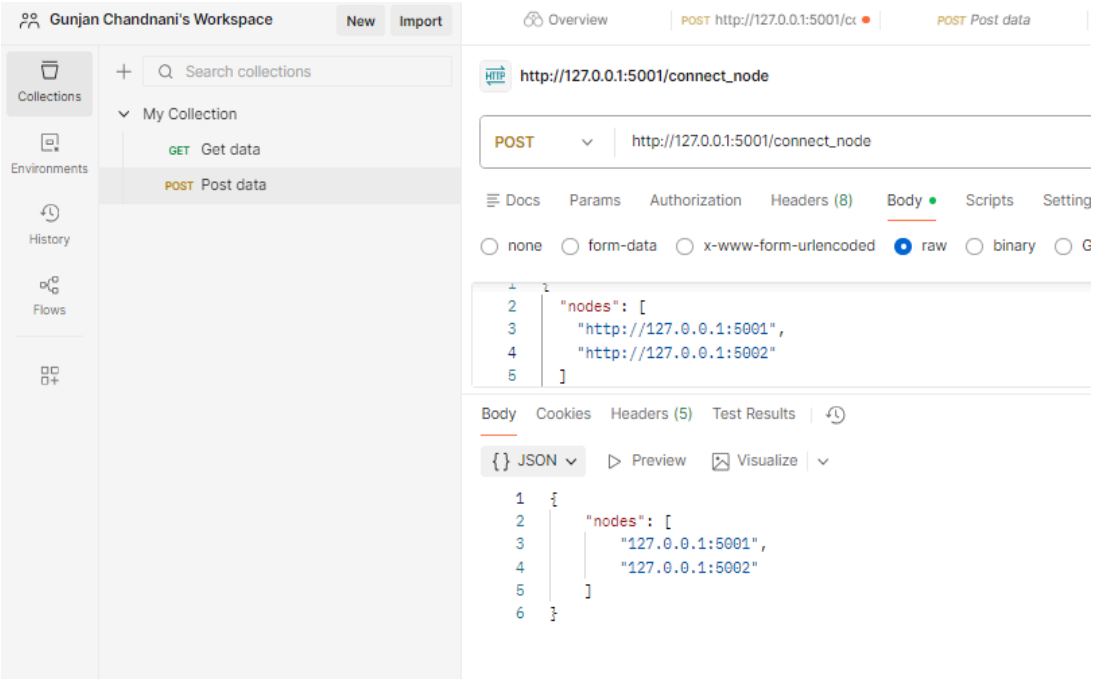
1. Connect Nodes (POST)

URL (from any node):

```
POST http://127.0.0.1:5001/connect_node
```

Body → raw → JSON

```
{
  "nodes": [
    "http://127.0.0.1:5002",
    "http://127.0.0.1:5003"
  ]
}
```



2. Add Transaction (POST)

POST - http://127.0.0.1:5001/add_transaction

http://127.0.0.1:5001/add_transaction

Save | Share

POST http://127.0.0.1:5001/add_transaction Send

Docs | Params | Authorization | Headers (8) | Body | Scripts | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON | Schema | Beautify

```
{
  "sender": "Gunjan",
  "receiver": "Chandnani",
  "amount": 50
}
```

Body | Cookies | Headers (5) | Test Results | 201 CREATED | 21 ms | 213 B

JSON | Preview | Visualize

```
{"message":"Transaction added to block 2"}
```

3. Mine Block (GET)

GET http://127.0.0.1:5001/mine_block

http://127.0.0.1:5001/mine_block

Save | Share

GET http://127.0.0.1:5001/mine_block Send

Docs | Params | Authorization | Headers (8) | Body | Scripts | Settings | Cookies

Body | Cookies | Headers (5) | Test Results | 200 OK | 11 ms | 464 B

JSON | Preview | Visualize


```
{
    "index": 2,
    "previous_hash": "9da4789f3a512a2f4174a70b8582fa13c116fe641e95abe7775a8880f055c47e",
    "proof": 533,
    "timestamp": "2026-02-06 12:19:32.314793",
    "transactions": [
        {
            "amount": 50,
            "receiver": "Chandnani",
            "sender": "Gunjan"
        },
        {
            "amount": 1,
            "receiver": "Richard",
            "sender": "27e8c5b543a940e3983c9148da34cae2"
        }
    ]
}
```

4. Get Blockchain (GET)
GET http://127.0.0.1:5001/get_chain
You'll see: ● Transactions inside blocks
● transactions: [] for new pending list

http://127.0.0.1:5001/get_chain

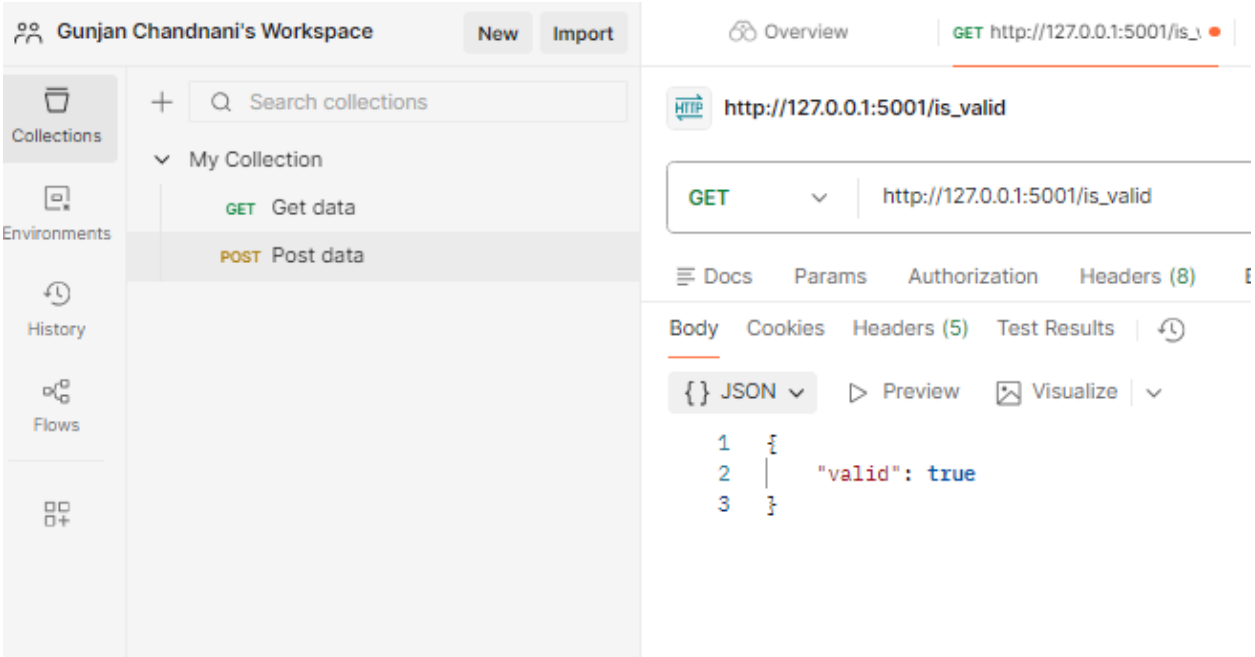
GET http://127.0.0.1:5001/get_chain

Docs Params Authorization Headers (8) Body Scripts Settings

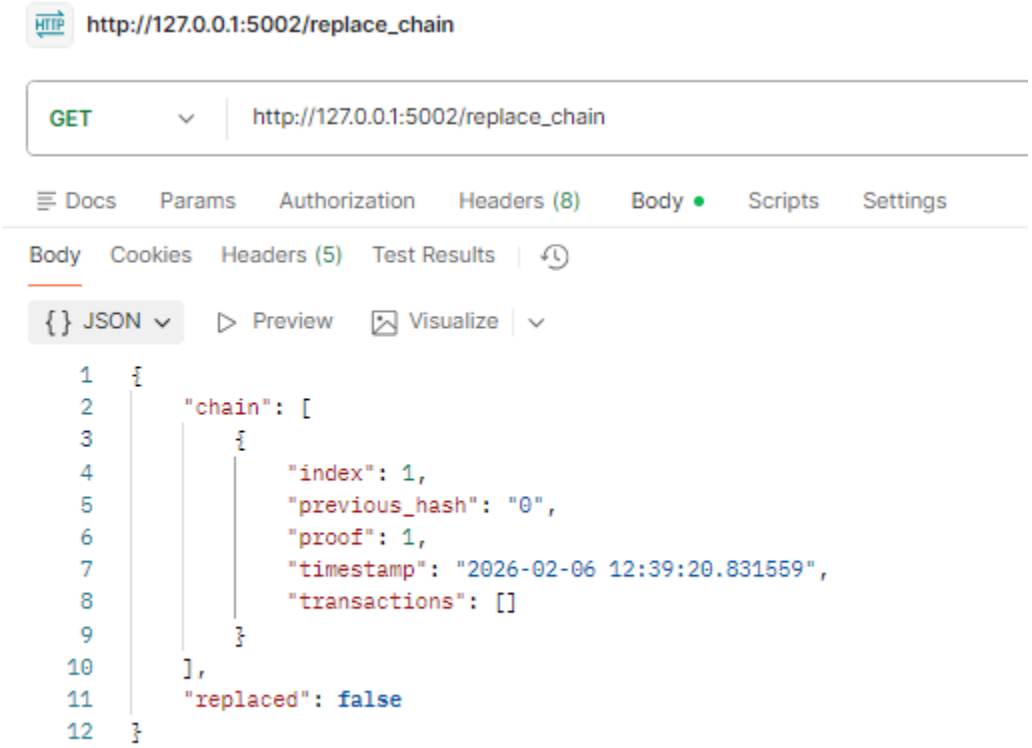
Body Cookies Headers (5) Test Results

{} JSON Preview Visualize


```
{
    "chain": [
        {
            "index": 1,
            "previous_hash": "0",
            "proof": 1,
            "timestamp": "2026-02-06 11:28:25.125567",
            "transactions": []
        },
        {
            "index": 2,
            "previous_hash": "9da4789f3a512a2f4174a70b8582fa13c116fe641e95abe7775a8880f055c47e",
            "proof": 533,
            "timestamp": "2026-02-06 12:19:32.314793",
            "transactions": [
                {
                    "amount": 50,
                    "receiver": "Chandnani",
                    "sender": "Gunjan"
                },
                {
                    "amount": 1,
                    "receiver": "Richard",
                    "sender": "27e8c5b543a940e3983c9148da34cae2"
```



5. Replace Chain (Consensus)
GET http://127.0.0.1:5002/replace_chain
Shorter chains get replaced by the longest valid one.



Conclusion

This blockchain system demonstrates how decentralization, cryptographic security, consensus mechanisms, and mining work together to create a secure and reliable distributed ledger. Through multi-node participation, Proof-of-Work mining, and chain replacement, the blockchain maintains integrity, consistency, and trust without